HTML-CSS-08 第8回_まとめ

第8回の目標

- **レスポンシブを思い出す**
- **より実践的なレスポンシブに慣れる**

レスポンシブの流れ（振り返り）

基本の流れは第3回で学んだ通り。 1. 今の見た目とデザインカンプを見比べる 2. 違いを見つける 3. コードに置き換えて書く

主な変更箇所もおさらいしておく： - 横並びの箇所を縦並びにする（またはその逆） - テキストの大きさ／改行を調節する - 要素の横幅／高さを調節する - 要素の表示／非表示を切り替える - 余白の大きさを調節する

- どの幅でも綺麗に見えるようにコーディングするのは実務で必須
- レスポンシブは「記述を上書きする」イメージ。上書きするときは、なるべく同じプロパティを使う
- 元々クラス名がないものは、HTMLで出てくる順番を見て「このあたり」という場所に書く

メディアクエリを書く場所（復習）

元のクラスの**直後**に書く。

書く場所	理由
元のクラスより**後（下）**	コードは上から読み込まれるため
元のクラスの**直後**	変更・修正するときに探しやすいため

レスポンシブの確認作業の流れ

1. まずは目安のブレイクポイント（1080px、768px）で作る
2. 320pxから2000pxくらいまで、少しずつ画面幅を広げて違和感がないか見る
3. 表示崩れを見つけたら対処する（ブレイクポイントを変えるか、CSSのプロパティで対応できるか考える）

- デザインにない部分も、こちらで判断して表示を整える（＝よしなに対応する）ことができると、ディレクターに喜ばれるコーダーになれる
- 数値は`Shift`＋上下矢印キーで10単位で動かせる（検証ツールなど）
- いろいろな画面幅を一気に確認できるChrome拡張機能もある（ Responsive Viewer）
- レスポンシブが効かないときは、`<meta name="viewport" ...>`が書けているか確認する

- 検証ツールの色の見方は第3回を参照

同じ詳細度で上書きする

レスポンシブでは、すでに指定されているスタイルをメディアクエリの中で上書きすることが多い。`:not(:first-child)`のような疑似クラス付きのセレクタを上書きしたいときは、必ずもともとと同じ書き方（同じ詳細度）で指定し直す必要がある。

```
/* ベース */
.sample_item:not(:first-child) {
  margin-top: 24px;
}

/* 上書きするときも同じセレクタの形で書く */
@media screen and (min-width: 1080px) {
  .sample_item:not(:first-child) {
    margin-top: 0;
  }
}
```

詳細度が異なるセレクタで上書きしようとすると、意図通りに反映されないことがあるので注意する。 - 例：`.sample_item:not(:first-child)`は擬似クラスがついている分、`.sample_item`より詳細度が高い。`.sample_item { margin-top: 0; }`では上書きできないので、同等以上の詳細度で書く - 🤔 CSSの詳細度を学ぼう - 今は「CSSには優先順位があり、詳細に指定するほど優先順位が高くなる」と知っておけばよい - 🤔 【余裕がある人向け】CSSの詳細度について

ショートハンドの上書きに注意する

`background`や`margin`のようなショートハンド（省略記法）プロパティは、1つのプロパティに複数の値がまとまっている。メディアクエリ内で一部の値だけ変えたいつもりでも、ショートハンドごと上書きすると、指定していない他の値まで初期値に戻ってしまうことがある。

```
/* ベース */
.sample {
  margin: 64px auto 0;
}

/* 上側の余白だけ変えたいのに、ショートハンドで書くと他の値も再指定が必要になる */
```

```
@media screen and (min-width: 1080px) {
  .sample {
    margin: 0 0 0; /* 本来残したい auto が消えてしまう例 */
  }
}
```

変えたい値だけを個別プロパティ（`margin-top`など）で指定し直す方が、意図しない上書きを避けやすい。

ショートハンド（復習）	意味
`margin: 上 左右 下;`	値が3つのとき
`margin: 上下 左右;`	値が2つのとき

margin: 0 auto; で中央寄せになる仕組み

1. `width`をつけると、余った分が`margin`として右側に大きく余る
2. 左右両方に`auto`をつけると、余った余白を左右均等に配分してくれる
3. 結果として中央寄せになる

- 中央寄せできる条件（ブロック要素であること、`width`か`max-width`があること）は第3回を参照
- 🖱 margin: auto;とは／CSSで中央寄せする方法まとめ

text-alignの影響範囲

- `text-align`は、当てた要素の**中のインライン要素**に影響する
- 「text」とあるが、テキストだけでなくインライン要素全体にかかるので注意

calc()で横幅を均等に分割する

画面幅によって「1行に並べる数」を変えたいとき、`calc()`を使うと余白込みで均等に横幅を分割できる。

```
/* 余白16pxを挟んで3列に均等分割する場合 */
.sample_item {
  width: calc((100% - 16px * 2) / 3);
}
```

- 余白の本数は「列数－1」になる点に注意する（3列なら間の余白は2本）
- 考え方：`（全体 − 余白の幅 × 余白の個数）÷ 横に並べたい個数（列数）`
  - 例：4列・余白16px → `calc((100% - 16px * 3) / 4)`

    - `calc((100% - 48px) / 4)`でも同じだが、`16px * 3`と書く方がどう計算したか分かり、修正や他の人が読むときに分かりやすい
- 演算子（`+` `-` `*` `/`）の前後には、必ず半角スペースを入れる
- 列数が増えると間の余白も増えるので、忘れずに調整する
- `100%`と書いてあるときは、何を基準にしているか親要素をたどる（例：親が`width: 32px;`なら、子の`width: 100%;`も32px）
- `justify-content: space-between;`で並べる方法と、`calc()`で幅まで計算してしまい`gap`で並べる方法の、どちらでも実現できる

画面幅で要素の表示・非表示を切り替える（ヘッダー）

SPではハンバーガーメニュー、PCではナビゲーションを横並びで表示する、という切り替えはレスポンシブの定番。

```
@media screen and (min-width: 1080px) {
  /* ハンバーガーメニューを非表示にする */
  .m_hamburger {
    display: none;
  }

  /* 隠れていたナビゲーションを表示して横並びにする */
  .l_header-nav {
    display: block;
  }

  .l_header-nav_list {
    display: flex;
    gap: 0 48px;
  }

  /* SPで縦並び用につけていた余白を、同じ詳細度で打ち消す */
  .l_header-nav_item:not(:first-child) {
    margin-top: 0;
  }
}
```

- 要素を画面幅によって表示・非表示させるときは、`display`（または`opacity`）で調整する

- `display: none;`の解除は、none以外ならどの値でも表示されるが、要素に合った値にする。迷ったら、そのHTMLタグのデフォルトの表示形式に戻す
- `flex`は、横並びにしたい要素の一番近くの親要素に当てる
- `gap`は「縦の余白 横の余白」の順で指定できる（`gap: 0 48px;`＝横方向だけ48px）。 gapを使って要素同士の余白を作る
    - `gap`は`flex`と必ずセットで使う

ヘッダーをレスポンシブ化する手順 1. ハンバーガーメニューを非表示にする 2. 隠れているナビゲーションメニューを表示する 3. ナビゲーションメニューを横並びにする 4. ナビゲーションの余白を調節する 5. ナビゲーションの文字の見た目を調整する

SPで透明にしたナビゲーションをPCで表示する

```
/* SP：透明にして、クリックもできないようにする */
.l_header-nav {
  opacity: 0;
  pointer-events: none;
}

@media screen and (min-width: 1080px) {
  /* PC：表示して、クリックできる状態に戻す */
  .l_header-nav {
    opacity: 1;
    pointer-events: auto;
  }

  /* SPで中央寄せに使っていたpositionを解除する */
  .l_header-nav_list {
    display: flex;
    position: static;         /* positionを解除 */
    transform: translate(0, 0);
    gap: 0 40px;
  }
}
```

- `opacity: 0;`だけだと要素は残っているので、透明なリンクに触れてページ遷移してしまう。`pointer-events: none;`でクリックできないようにしておく
- `position: static;`で`position`を解除できる
- `opacity`は、フェードイン・フェードアウトのようにゆっくり表示・非表示させるアニメーションで使う

要素を見えなくするプロパティの違い

プロパティ	見える？	存在する？	触れる（クリックできる）？	特徴
`opacity: 0;`	見えない	いる	触れる	透明マントのような状態。上に張り付くので、下の要素をクリックできなくなる
`visibility: hidden;`	見えない	いる	触れない	透明になり、クリックもできなくなる
`display: none;`	見えない	いない	触れない	存在そのものがなくなり、下や右の要素が詰まってくる

- 要素を見えなくするCSSプロパティの違い／CSSで要素を透明化する方法3選
- 検証ツールを使うと、適用されているスタイルの確認や、その場での仮編集ができる。検証ツールの使い方

ブレイクポイントは柔軟に決める

- 基本は768px／1080pxだが、横並びにすると窮屈になる箇所は**1200px**にするとやりやすい（KV・お悩みセクション・事例ページなど）
  - 理由：コンテンツ幅が最大1088pxなので、1080px付近の画面幅だと左端にぴったりくっつく表示崩れが起きるため
- レスポンシブは、少しずつ画面幅を広げながら違和感があるところを直していく
- 【＋α】余裕があれば、768px（中間幅）でも綺麗に表示されるように書いてみる

KVのテキストを左寄せにする（transformの上書き）

SPで上下左右中央寄せにしていた要素を、PCで左寄せにしたいときは、`left`と`transform`の両方を上書きする。

```
@media screen and (min-width: 1080px) {
  .m_kv-box {
    left: 16px;
    transform: translate(0, -50%); /* 縦方向の中央寄せだけ残す */
    text-align: left;
  }
}
```

- `transform: translate(-50%, -50%)` のX軸を `0` に戻すことで、縦の中央寄せは残したまま横方向だけ左寄せにできる（第5回の「基準点からのずらし方」参照）
- 中央寄せのために `margin: 40px auto 0;` としていたボタンも、PCでは `margin: 64px 0 0;` のように左右の `auto` を外して左寄せにする
- KVのボックスに `max-width: var(--content-width-lg);` をつける理由：`position: absolute;` がついていると要素が浮き、paddingで空けたい余白の部分まで要素が寄ってしまうため。デザイン通りにするために書いている

交互に左右を入れ替えるレイアウト

画像とテキストが交互に左右入れ替わるデザインは、`nth-child(2n)` で偶数番目だけ配置を変える。

```
@media screen and (min-width: 1080px) {
  /* 偶数番目は画像を左に */
  .sample_info:nth-child(2n) .sample_info_img-wrapper {
    left: 0;
  }

  /* 偶数番目はテキストを右に寄せる */
  .sample_info:nth-child(2n) .sample_info_content {
    margin-left: auto;
  }
}
```

- `flex` で並べている場合は、`flex-direction: row-reverse;` で並び順を逆にする方法もある（事例ページで使用）
- `margin-left: auto;` をかけると、flexアイテムや固定幅の要素を右端に寄せられる

【応用】画像を画面の端まで広げる

```
.sample_info_img-wrapper {
  position: absolute;
  top: 0;
  right: 0;
  width: calc(100vw - (343px + 120px) - (100vw - var(--content-width-lg)) / 2);
}
```

- 「画面幅（`100vw`）」から「テキストの幅＋余白」と「コンテンツ幅の外側の余白（片側分）」を引くことで、画像だけをコンテンツ幅の外（画面の端）まで広げている

横スクロールをPCで解除する

SPで`overflow-x: scroll;`にしていた要素は、PCで全部収まる場合はスクロールを解除し、幅も通常に戻す。

```
@media screen and (min-width: 1080px) {
  .service_menu_wrapper {
    overflow: inherit; /* 親要素の値を引き継ぎ、スクロールを解除 */
    width: var(--content-width);
  }

  .service_menu_list {
    width: 100%; /* calc()で広げていた幅を戻す */
  }
}
```

フレキシブルレイアウトという考え方

デザインカンプには載っていない画面幅（スマホとタブレットの間など）でも、レイアウトが崩れずに綺麗に見えるようにコーディングする考え方。レイアウトの変え方自体に厳密な正解はなく、次のポイントを押さえていれば自由とされている。 - 変更に強く、保守性が高い - 不必要に長くない - フレキシブルレイアウトである（決まった画面幅以外でも崩れにくい）

デザインデータからコーディング！ブログサイトのレスポンシブ化（フレキシブルレイアウトの解説あり）

コーディングを効率化するスニペット登録

エディタにメディアクエリのスニペット（`@lg`や`@sm`と打つと展開される定型コード）を登録しておくと、同じ記述を何度も打たずにコーディングを効率化できる。人によって登録した打ち方は異なるので、自分の環境でどう登録したか覚えておくとよい。 - style.cssで`@lg`＋Enter、`@sm`＋Enterと打って、スニペットが出るか確認する（登録方法は第3回参照）

【卒業後に学ぼう】

- コーディング時のルールや思想（全30項目）：いろいろなコーダーの考え方を読んでおくと、何を意識すべきか分かる
- ググり力はエンジニアに必須の能力
- レスポンシブコーディングの仕組み

次回（JavaScript）に向けて

- 次回からJavaScriptの授業になる。今までと違う言語なので、分からないのが普通
- 予習課題だけは必ずやってくる（やらないと授業2時間の情報量でパンクする）。余裕があれば授業の演習を1周しておくと理解度が上がる
- 予習記事は13本あり全部読むのは難しいので、「予習課題 → 分からないところの記事を読む → 分からない箇所を質問フォーム・勉強会で聞く」の流れで進める

本日のまとめ

1. レスポンシブの基本の流れは、これまでの回と変わらない（デザインと見比べる→違いを見つける→CSSで書く）
2. メディアクエリ内で上書きするときは、元のセレクタと同じ詳細度で書く
3. ショートハンドプロパティは、一部だけ変えたいときに他の値まで上書きしてしまうことがある
4. `calc()`を使うと、余白を考慮した均等な横幅分割ができる
5. デザインカンプにない画面幅でも崩れない「フレキシブルレイアウト」を意識する
6. 要素の表示・非表示は`display`で切り替え、要素同士の余白は`gap`でまとめて作れる
7. ブレイクポイントは768px／1080pxに限らず、必要に応じて1200pxなども使う

参考記事

- 様々なデバイスに対応するためにレスポンシブを学ぼう
- デザインデータからコーディング！ブログサイトのレスポンシブ化
- デザインデータからコーディング！お問合せフォームのレスポンシブ化
- flexboxを使いこなそう
- gapを使って要素同士の余白を作る
- 検証ツールの使い方（仮編集なども行える）
- フレキシブルレイアウト
- CSSの詳細度について
- margin: auto;とは
- CSSで中央寄せする方法まとめ